数据结构

排序

考试大纲

七、排序

（一）排序的基本概念

（二）直接插入排序

（三）折半插入排序

（四）起泡排序（bubble sort）

（五）简单选择排序

（六）希尔排序（shell sort）

（七）快速排序

（八）堆排序

（九）二路归并排序（merge sort）

（十）基数排序

（十一）外部排序

（十二）排序算法的分析和应用

1 排序的基本概念



稳定性 (Stability)

若 $R_i = R_j$ 且 R_i 在 R_j 之前，排序后顺序不变则为稳定。



内/外部排序

内部排序：数据全在内存中。

外部排序：数据量大，需借助外部存储。



核心指标

时间复杂度、空间复杂度、比较次数、移动次数。

排序算法性能预览

表 8.1　各种排序算法的性质

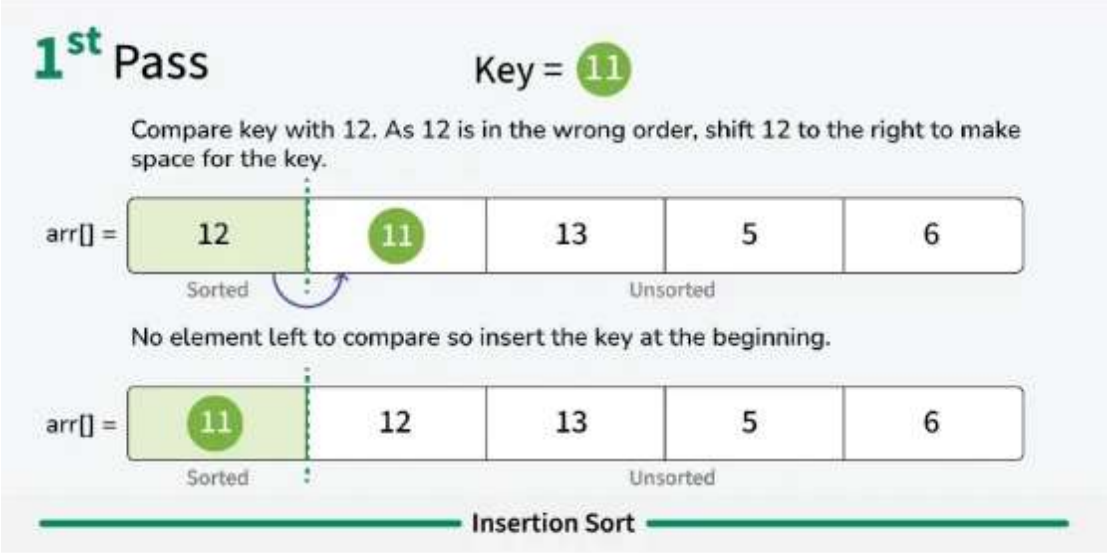
算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

2 直接插入排序

算法逻辑

将待排序记录划分为“已排序”和“未排序”两部分。每次从未排序序列取出一个记录插入到已排序序列的正确位置。

```
L.data[0] = L.data[i]; // 哨兵
for(j=i-1; L.data[0]   L.data[j+1] = L.data[j];
L.data[j+1] = L.data[0];
```



直接插入排序 – 性能分析

- **稳定性：** 稳定。
- **比较次数：** 最好 n–1 次，最坏 $\approx n^2/2$ 次。
- **移动次数：** 最好 0 次，最坏 $\approx n^2/2$ 次。
- **适用性：** 适用于 n 较小或基本有序的情况。

> 🚨 **关键考点：** 在直接插入排序中，若初始序列基本有序，则效率接近 O(n)。

3 折半插入排序

核心优化：减少比较次数

利用已排序序列是有序的特性，通过**折半查找**确定插入位置。

```
while(low <= high){
  mid = (low + high)/2;
  if(L.data[mid] > L.data[0]) high = mid - 1;
  else low = mid + 1;
}
```

性能变化： 比较次数从 $O(n^2)$ 降低到 $O(n \log_2 n)$。

局限性： 移动次数仍为 $O(n^2)$，总时间复杂度不变。

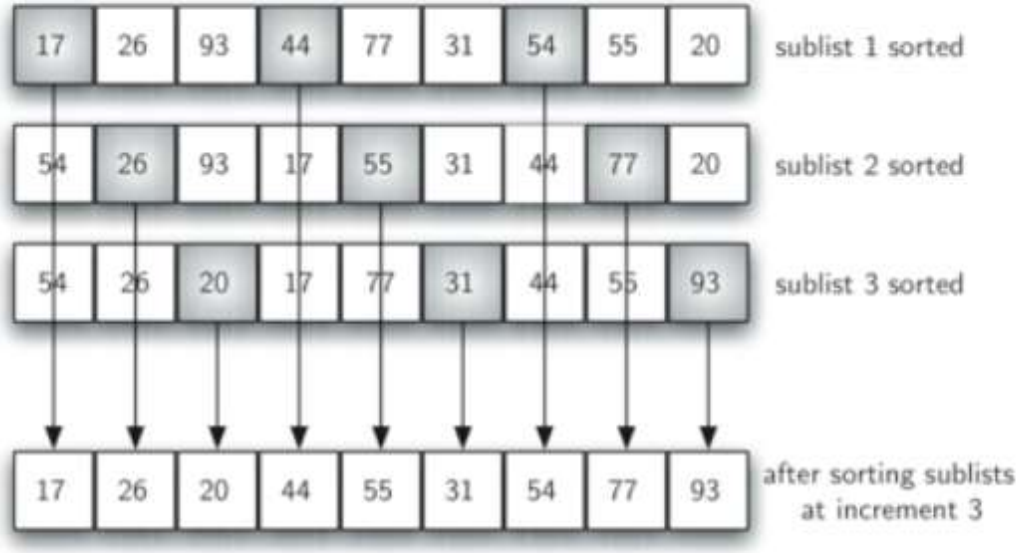
4 希尔排序 (Shell Sort)

算法思想

又称“缩小增量排序”。先将整个待排序序列分割成若干个子序列分别进行直接插入排序。

增量序列： 通常取 $d_1 = n/2, d_{i+1} = d_i/2$ 直至 d_k = 1。

🚨 注意：希尔排序是不稳定的！



5 冒泡排序 (Bubble Sort)

交换排序的一种。从前往后两两比较，若逆序则交换。

```
for(i=0; i
  flag = false;
  for(j=n-1; j>i; j--)
    if(L.data[j-1] > L.data[j]){ Swap(L, j-1, j); flag = true; }
  if(flag == false) return;
}
```

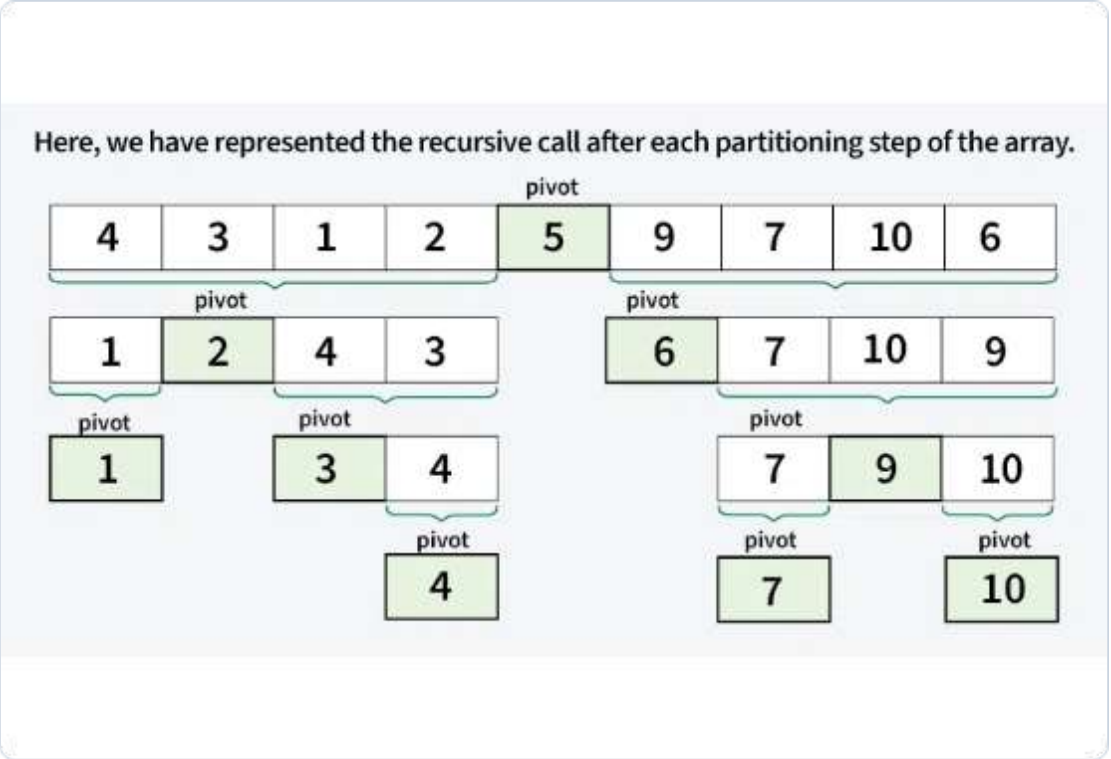
关键结论： 每一趟都会将一个“极值”移动到最终位置。若一趟排序中没有发生交换，说明序列已然有序。

6 快速排序 (Quick Sort)

分治法 (Divide & Conquer)

最常考的内部排序算法。平均性能最优。

1. 选择枢轴 (Pivot)。
2. 划分 (Partition)：左小右大。
3. 递归处理左右子序列。



快速排序：性能与深度分析

递归树： 快排的过程可看作一棵递归树。若划分均衡，树深 $O(\log_2 n)$；若极端不均，树深 O(n)。

空间复杂度

最好：$O(\log_2 n)$(递归栈深度)
最坏：O(n)

时间复杂度

最好/平均：$O(n \log_2 n)$
最坏：$O(n^2)$ (序列已序或逆序)

🚨 **注意：** 枢轴的选择决定了快排的效率。选取“三数取中”或随机枢轴可避免最坏情况。

7 简单选择排序

每一趟从待排序序列中选出最小的，与未排序部分的第一个记录交换。

```
for(i=0; i
  min = i;
  for(j=i+1; j
  if(min != i) Swap(L, i, min);
}
```

- **稳定性：** 不稳定。
- **时间复杂度：** 无论初始状态如何，均为 $O(n^2)$。
- **移动次数：** 较少，最好 0 次，最坏 3(n-1) 次。

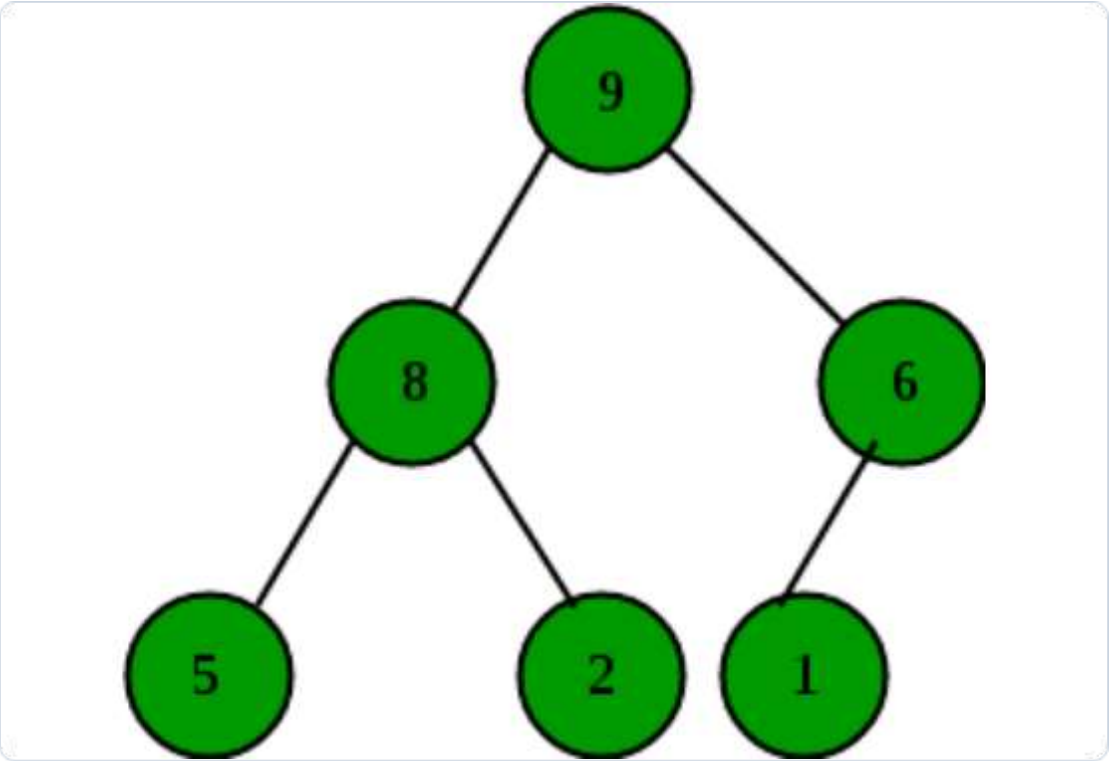
8 堆排序 (Heap Sort)

堆的定义

完全二叉树的顺序存储。其满足：

- **大根堆：** $L[i] \geq L[2i]$且 $L[i] \geq L[2i+1]$。
- **小根堆：** $L[i] \leq L[2i]$且 $L[i] \leq L[2i+1]$。

根节点必为序列中的最大(或最小)值。



堆排序核心：建堆与调整

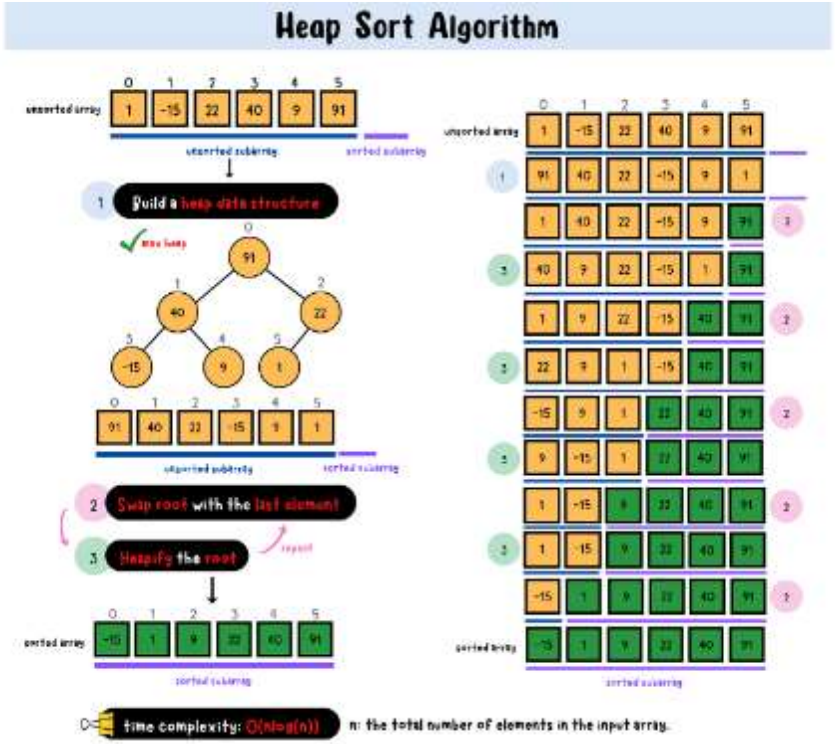
1. 初始建堆 (BuildHeap)

从最后一个非叶节点 $\lfloor n/2 \rfloor$开始，自底向上进行**筛选(HeadAdjust)**。

复杂度： O(n)。

2. 排序过程

将堆顶(最大值)与堆底元素交换，输出堆顶。对剩余 n-1 个元素重新调整为堆。重复执行。



Heap-Sort

Sorting Strategy:

1. Build Max Heap from unordered array;

2. Find maximum element A[1];

3. Swap elements A[*n*] and A[1]:
   now max element is at the end of the array!

4. Discard node *n* from heap
   (by decrementing heap-size variable)

5. New root may violate max heap property, but its children are max heaps. Run max_heapify to fix this.

6. Go to Step 2 unless heap is empty.

堆排序性能分析

- **稳定性：** 不稳定。
- **时间复杂度：** $O(n \log_2 n)$。各阶段均平稳。
- **空间复杂度：** O(1) (原地排序)。

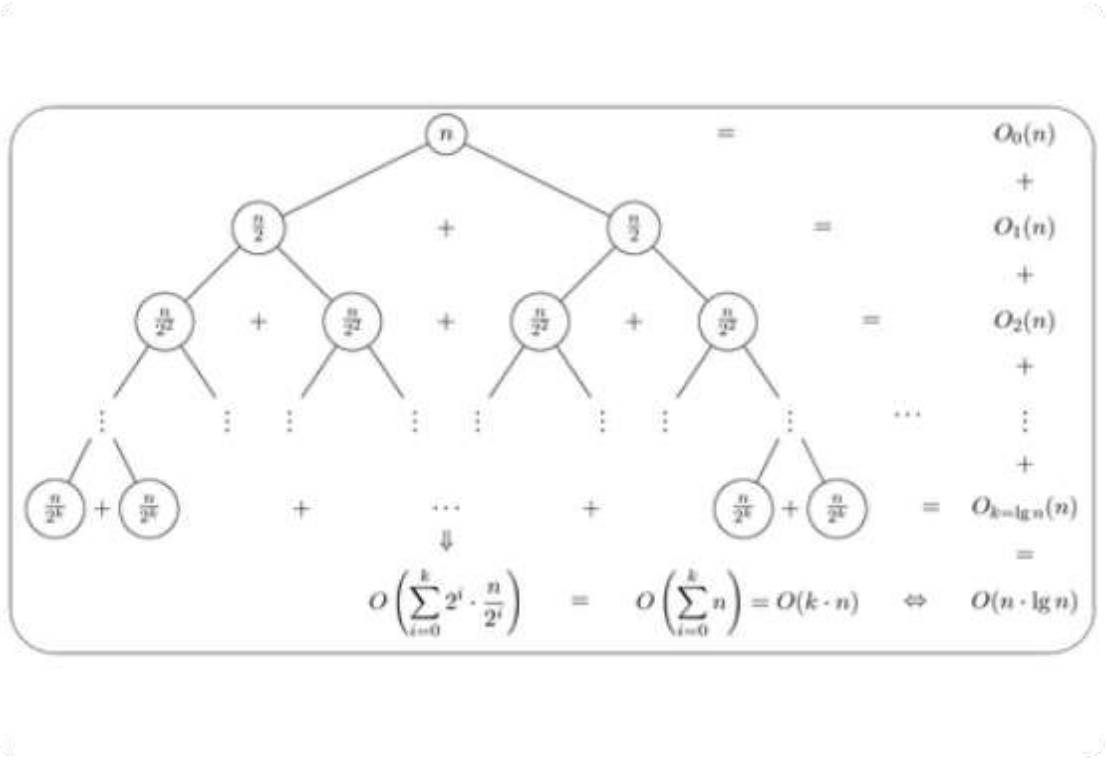
> **提醒：** 堆排序非常适合从海量数据中选出前 K 个最大值 (Top-K 问题)。

9 二路归并排序 (Merge Sort)

分治策略

将序列划分为两个子序列，分别归并排序，最后将两个有序表合并。

合并操作： 需要辅助空间 O(n)。



归并排序 – 性能分析

- **稳定性：** 稳定。
- **时间复杂度：** $O(n \log_2 n)$。
- **空间复杂度：** O(n)。

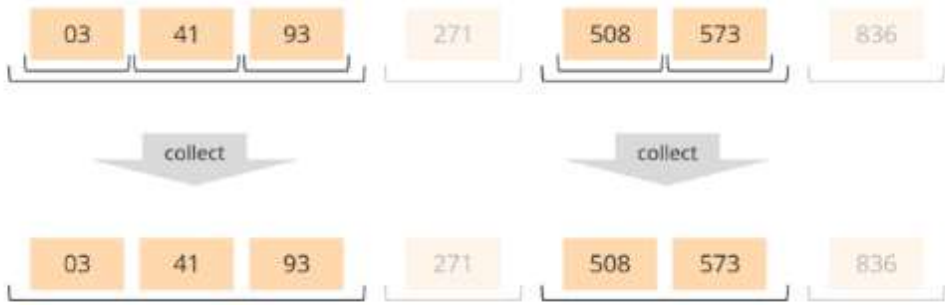
归并排序是考纲中空间复杂度最高的主流内部排序算法。它对于链式存储同样有效。

7.10 基数排序 (Radix Sort)

非比较排序

通过对关键字的各位进行“分配”和“收集”来实现。

- **LSD (最低位优先)：** 从个位开始。
- **MSD (最高位优先)：** 从最高位开始。



基数排序性能分析

设 n 个记录，关键字位数为 d，基数为 r (如十进制 r=10)：

- **时间复杂度：** O(d(n+r))。
- **空间复杂度：** O(r)。
- **稳定性：** 稳定。

适用场景： n 很大，但 d 较小，且 r 固定 (如身份证号、日期排序)。

算法选择：基于数据规模 n

n 较小

直接插入或简单选择。若考虑稳定性，用直接插入。

n 较大

快排、堆排或归并。快排平均最快；堆排空间最省；归并最稳。

基本有序

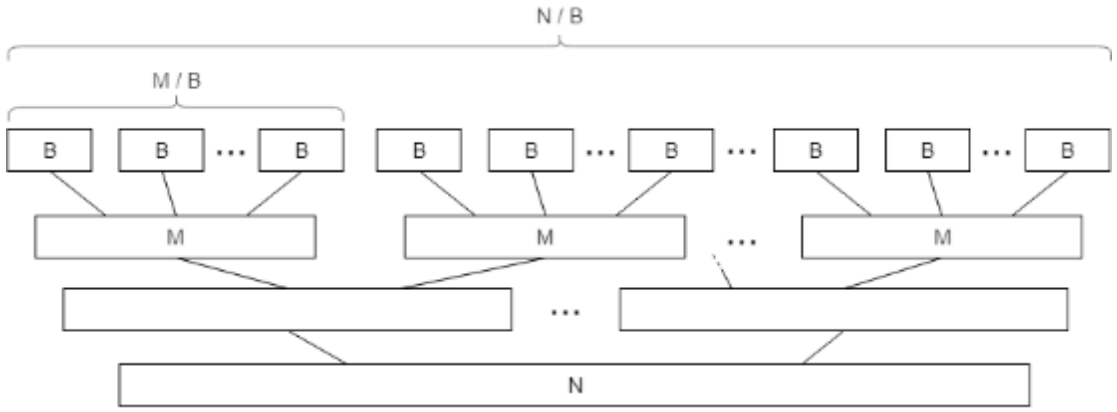
直接插入或冒泡。快排效率反而极低。

11 外部排序 (External Sorting)

背景与需求

当待排序数据量极大，不能一次装入内存。需要借用磁盘空间。

核心开销： 磁盘 I/O 次数。总时间 = 内部排序 + 外存读写 + 内部归并。



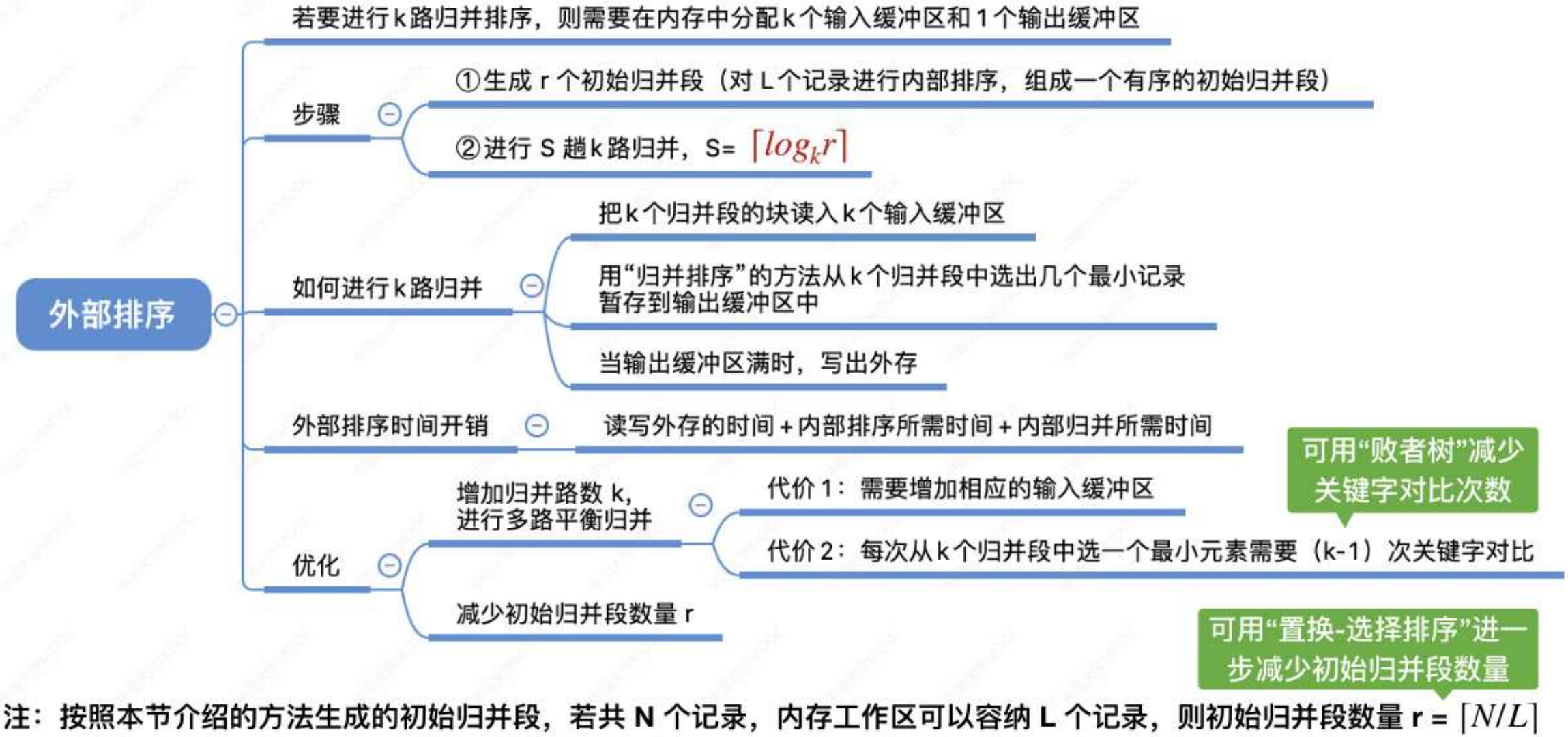
多路平衡归并

为了减少归并趟数 S，可以增加归并路数 k。

$$\text{归并趟数 } S = \lceil \log_k m \rceil$$

增加 k 可以减少 S，从而减少 I/O 次数。但增加 k 会导致内部归并比较次数增加。

解决方案： 引入**败者树 (Loser Tree)**，使内部归并时间与 k 无关。



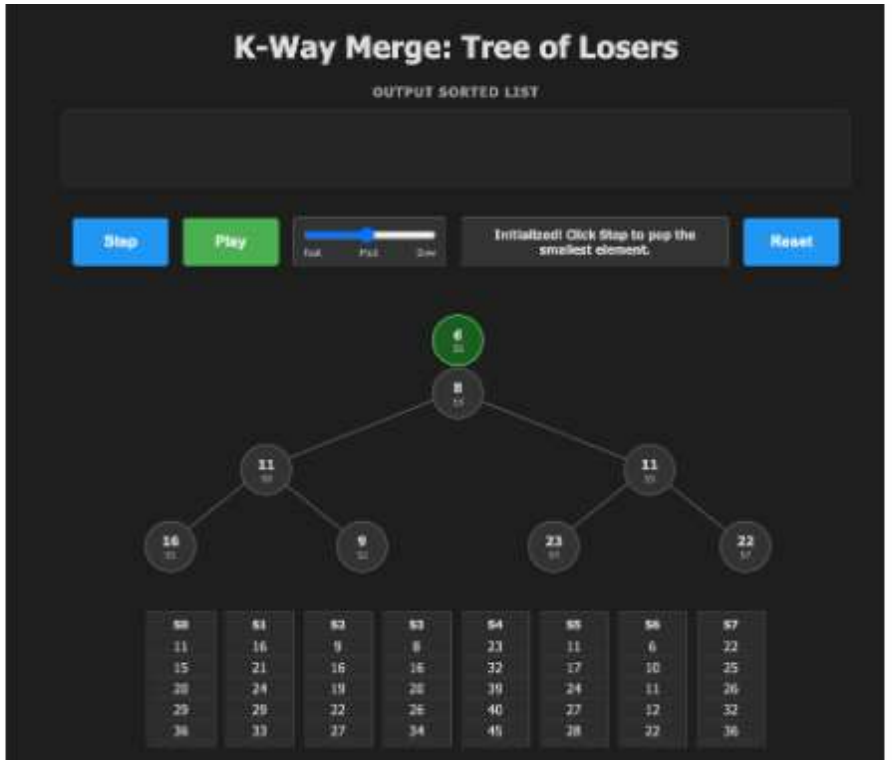
注：按照本节介绍的方法生成的初始归并段，若共 N 个记录，内存工作区可以容纳 L 个记录，则初始归并段数量 r = $\lceil N/L \rceil$

败者树 (Loser Tree)

逻辑原理

类似于锦标赛。叶子节点存数据，非叶子节点存“败者”索引，根上方设一个冠军位置。

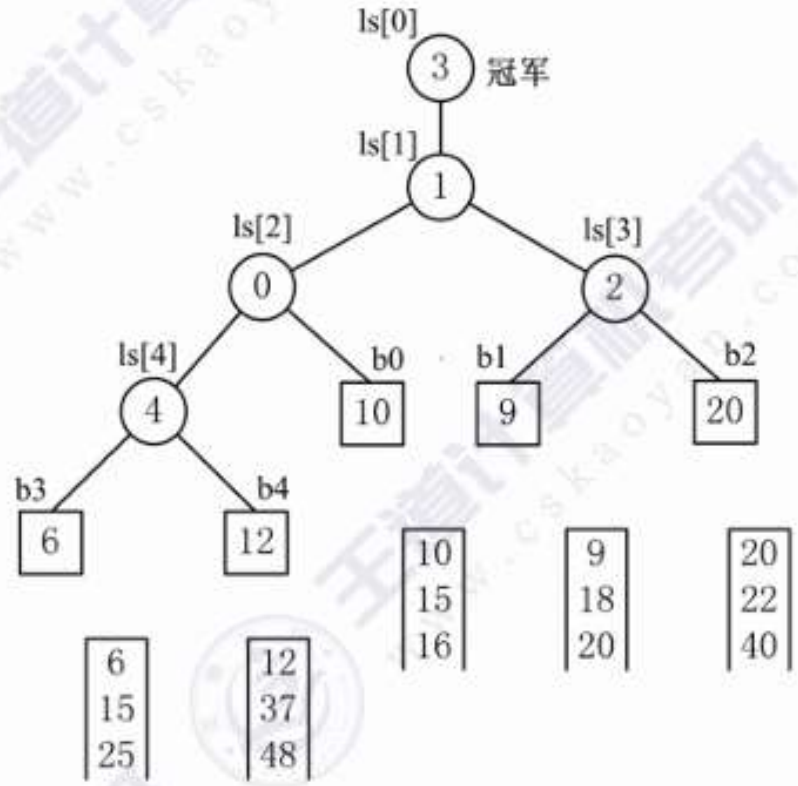
优势： 每次取出一个最小值后，调整路径长度仅为 $\lceil \log_2 k \rceil$。



败者树解决的问题：使用多路平衡归并可减少归并趟数，但是用老土方法从 k 个归并段选出一个最小/最大元素需要对比关键字 k-1 次，构造败者树可以使关键字对比次数减少到 $\lceil log_2 k \rceil$

败者树可视为一棵完全二叉树（多了一个头头）。k个叶结点分别对应 k 个归并段中当前参加比较的元素，非叶子结点用来记忆左右子树中的“失败者”，而让胜者往上继续进行比较，一直到根结点。

如何构造和使用败者树？——看图记忆



置换-选择排序 (Replacement-Selection)

目的：生成更长的**初始归并段**，从而减少归并趟数。

1. 从输入流读取 w 个记录入堆。
2. 选最小记录 min 输出，读取新记录 next。
3. 若 next > min，则 next 参与本段排序；若 next < min，则 next 等待下一段。

结论： 初始归并段的平均长度可达内存工作区大小的 2 倍。

设初始待排文件为FI，初始归并段输出文件为FO，内存工作区为WA，FO和WA的初始状态为空，WA可容纳w个记录。置换-选择算法的步骤如下：

1）从FI输入w个记录到工作区WA。

2）从WA中选出其中关键字取最小值的记录，记为MINIMAX记录。

3）将MINIMAX记录输出到FO中去。

4）若FI不空，则从FI输入下一个记录到WA中。

5）从WA中所有关键字比MINIMAX记录的关键字大的记录中选出最小关键字记录，作为新的MINIMAX记录。

6）重复3）～5），直至在WA中选不出新的MINIMAX记录为止，由此得到一个初始归并段，输出一个归并段的结束标志到FO中去。

7）重复2）～6），直至WA为空。由此得到全部初始归并段。

置换–选择排序

初始归并段输出文件 FO：

MINIMAX=30

归并段1：

4	6	7	9	11	13	14	16	22	30

若WA内的关键字都比 MINIMAX 更小，则该归并段在此截止

2
10
3

内存工作区WA

初始待排序文件 FI：

	19	20	17	1	23
5	36	12	18	21	39

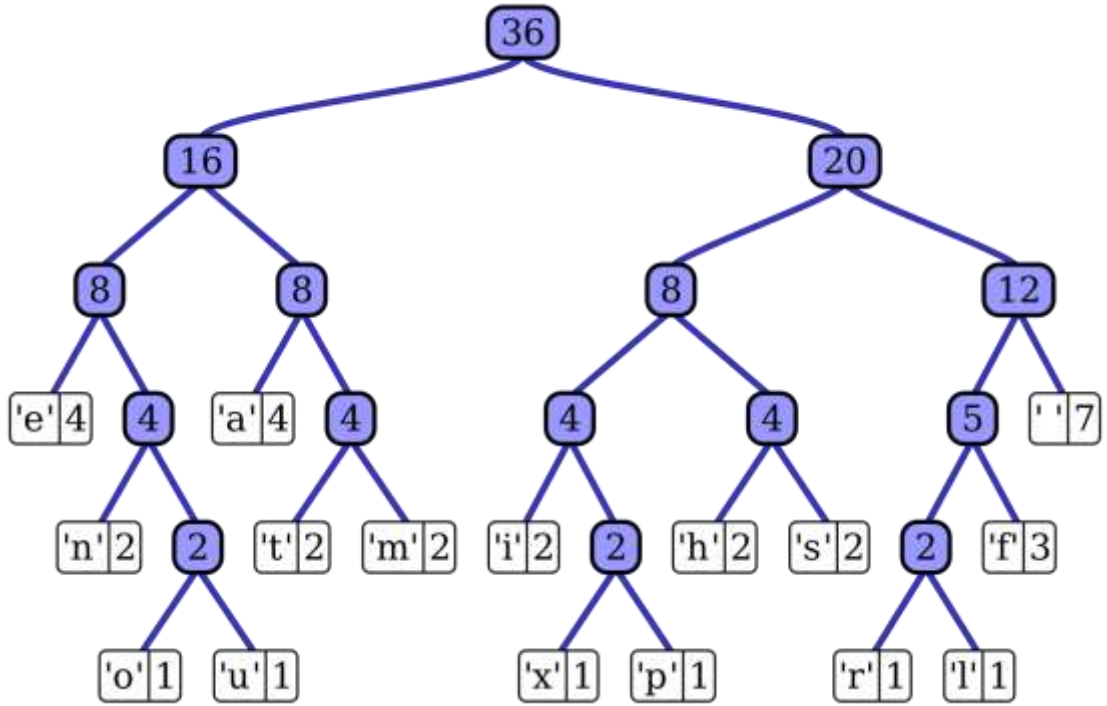
注：假设用于内部排序的内存工作区只能容纳3个记录

最佳归并树 (Optimal Merge Tree)

哈夫曼树的应用

若初始归并段长度不等，如何组织归并顺序使 I/O 最少？

方法： 建立以归并段长度为权值的 k 叉哈夫曼树。



🚨 **注意：** 若 $(m-1)\%(k-1) \neq 0$，需要补充“虚段”(权值为0)。

典型例题 1

题目：在下列排序算法中，哪一个在最坏情况下的时间复杂度为 O(n^2)，且它是稳定的？

A. 快速排序　B. 堆排序　C. 直接插入排序　D. 简单选择排序

答案解析：

快速排序最坏 $O(n^2)$ 但不稳；堆排序最坏 $O(n\log n)$；简单选择不稳。

直接插入排序最坏 $O(n^2)$且**稳定**。正确答案：C。

典型例题 2

题目：设 n=1000，若要求以最快的速度选出前 10 个最大的元素，最合适的排序方法是？

答案解析：

对于 Top-K 问题，**堆排序**最为理想。只需建立一个大小为 10 的小根堆，后续 990 个元素逐一比较即可。时间复杂度 $O(n \log k)$。

21. 【2018 统考真题】对初始数据序列(8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6)进行希尔排序。若第一趟排序结果为(1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8)，第二趟排序结果为(1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9)，则两趟排序采用的增量（间隔）依次是（　）。

A. 3, 1　　B. 3, 2　　C. 5, 2　　D. 5, 3

21. D

如下图所示。



第一趟分组：8, 1, 6；3, 4；9, 7；11, 5；2, 10；间隔为 5，排序后组内递增。

第二趟分组：1, 5, 4, 10；3, 2, 9, 8；7, 6, 11；间隔为 3，排序后组内递增。

因此，选择选项 D。

【2019 统考真题】排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是（ ）。

A. 5, 2, 16, 12, 28, 60, 32, 72　　B. 2, 16, 5, 28, 12, 60, 32, 72

C. 2, 12, 16, 5, 28, 32, 72, 60　　D. 5, 2, 12, 28, 16, 32, 72, 60

18. D

基于上题中分析得出的结论，观察题中的四个选项，选项 A 的 28, 72 符合第一种情况；选项 B 的 2, 72 符合第一种情况；选项 C 的 2, 28, 32 符合第一种或第二种情况；最后看选项 D，只有 12 和 32 处于最终位置，既不符合第一种情况，又不符合第二种情况。

19.【2023 统考真题】使用快速排序算法对数据进行升序排序，若经过一次划分后得到的数据序列是 68, 11, 70, 23, 80, 77, 48, 81, 93, 88，则该次划分的枢轴是（ ）。

A. 11　　B. 70　　C. 80　　D. 81

19. D

第一趟划分后得到的序列中只有一个枢轴，因此可将当前序列和最终排好序的序列进行比较，如下表所示。枢轴会出现在两个序列的相同位置，可以看出枢轴只可能是 77、81，选项只有 81。在当前序列中，77 左边有比它大的元素 80，因此 77 不是枢轴；而 81 左边都是比它小的元素，右边都是比它大的元素，因此 81 是枢轴。

当前序列	68	11	70	23	80	77	48	81	93	88
最终序列	11	23	48	68	70	77	80	81	88	93

18. 【2018 统考真题】在将序列(6, 1, 5, 9, 8, 4, 7)建成大根堆时，正确的序列变化过程是(　)。

A. 6, 1, 7, 9, 8, 4, 5 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

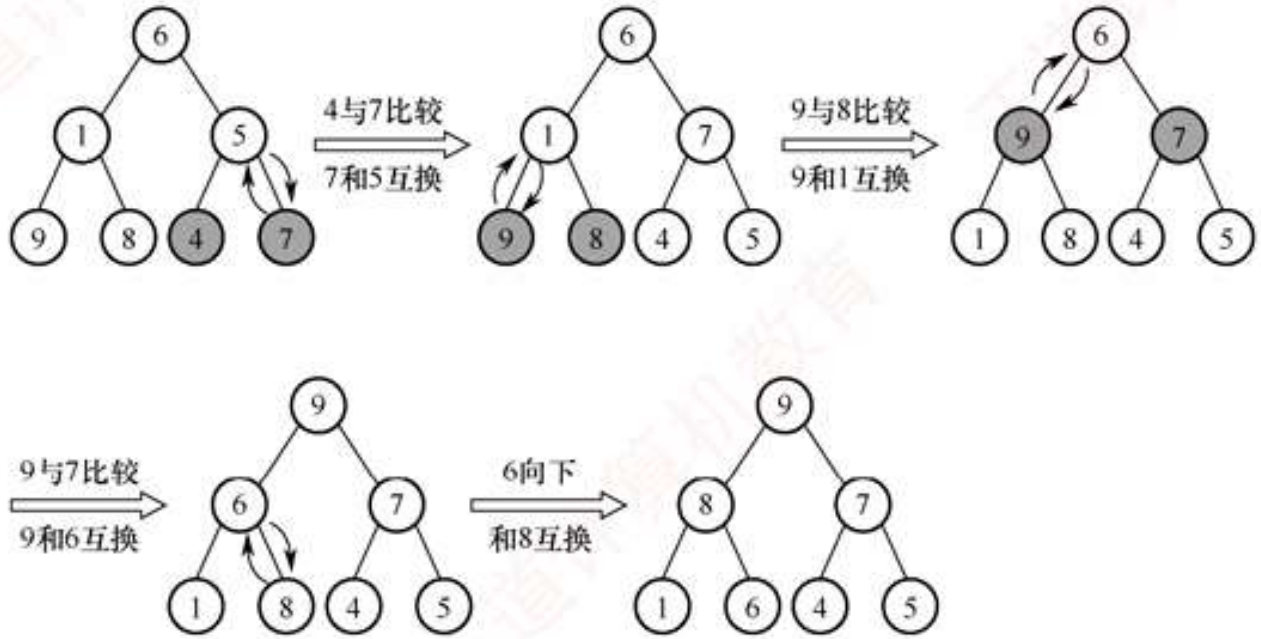
B. 6, 9, 5, 1, 8, 4, 7 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

C. 6, 9, 5, 1, 8, 4, 7 → 9, 6, 5, 1, 8, 4, 7 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5

D. 6, 1, 7, 9, 8, 4, 5 → 7, 1, 6, 9, 8, 4, 5 → 7, 9, 6, 1, 8, 4, 5 → 9, 7, 6, 1, 8, 4, 5 → 9, 8, 6, 1, 7, 4, 5

18. A

要熟练掌握建堆和调整堆的方法，从序列末尾开始向前遍历，变换过程如下图所示。

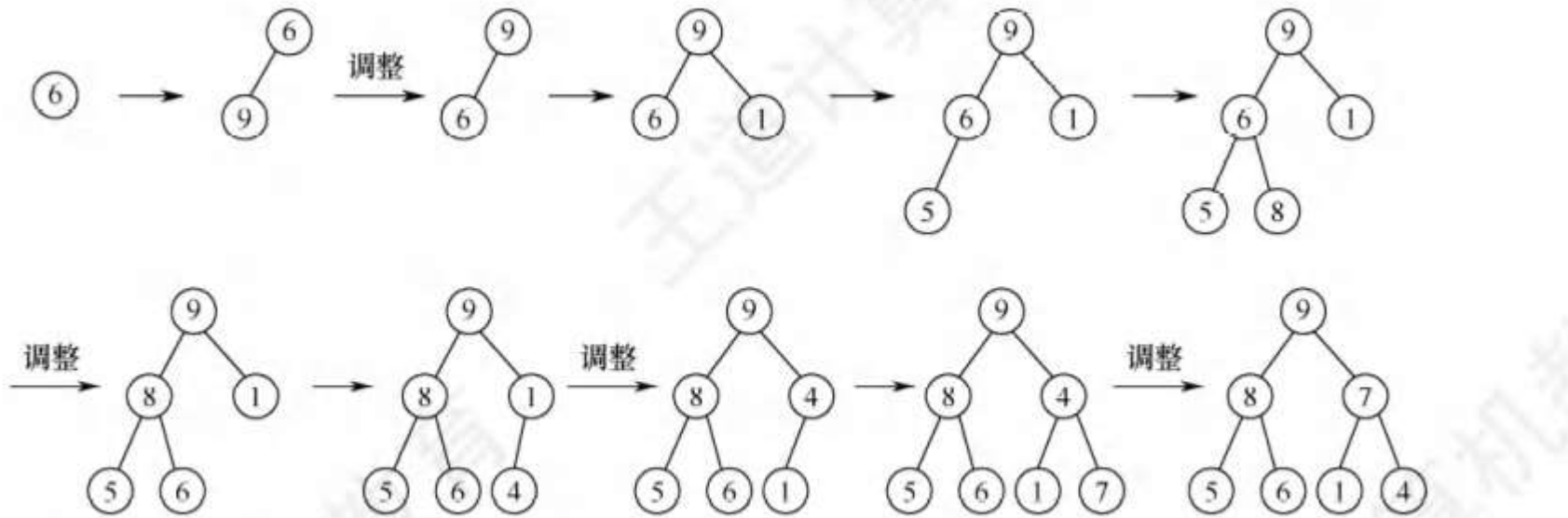


20.【2021 统考真题】将关键字 6,9,1,5,8,4,7 依次插入初始为空的大根堆 H，得到的 H 是(　)。

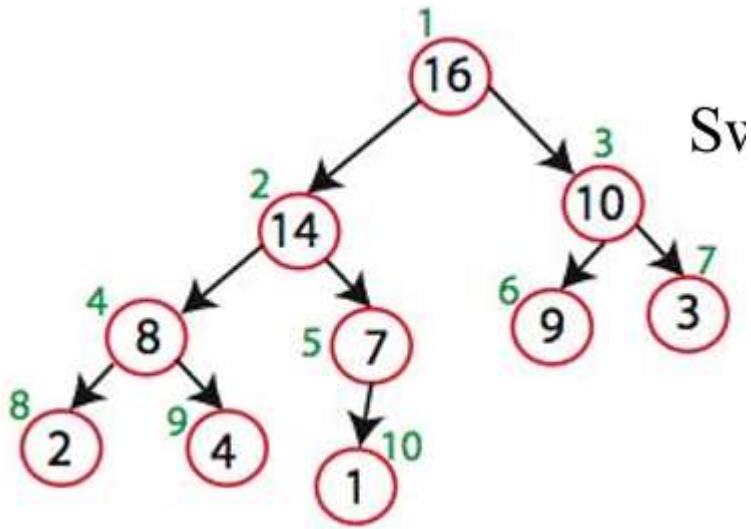
A. 9,8,7,6,5,4,1　　B. 9,8,7,5,6,1,4　　C. 9,8,7,5,6,4,1　　D. 9,6,7,5,8,4,1

20. B

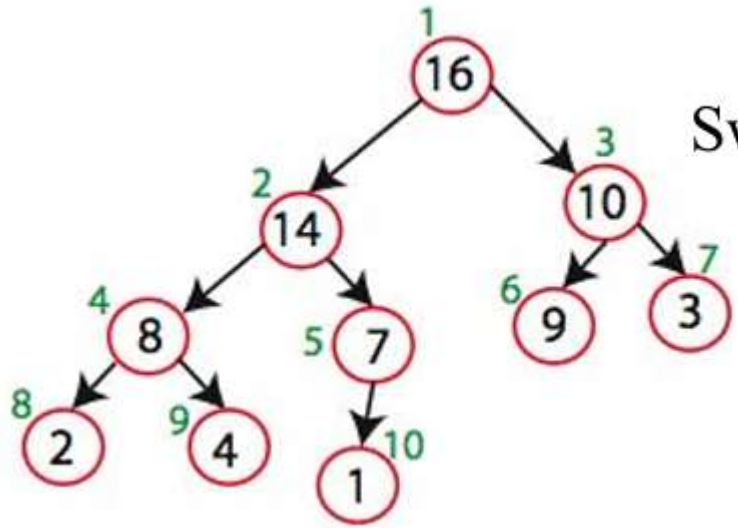
要熟练掌握调整堆的方法，建堆的过程如下图所示，所以答案选择选项 B。



注意，给定序列依次插入空堆的结果与给定序列直接调整成堆的结果是不同的，最终得到的堆的形式也不同。若对序列 6, 9, 1, 5, 8, 4, 7 直接调整成堆，则会误选 C 项。

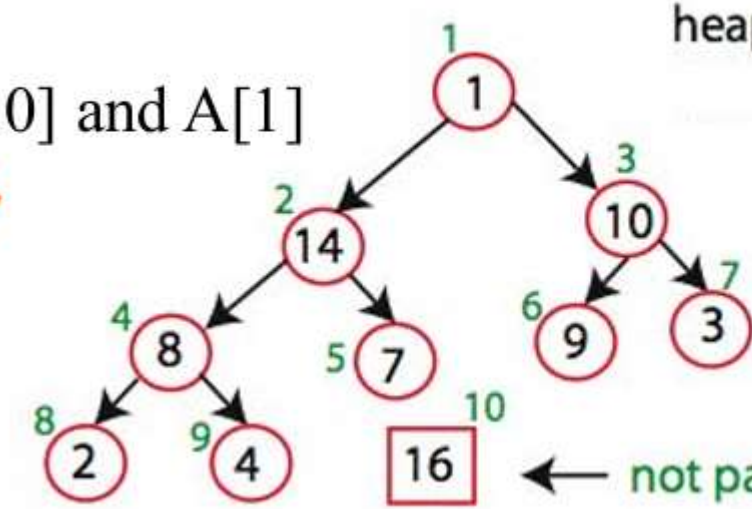


Sv



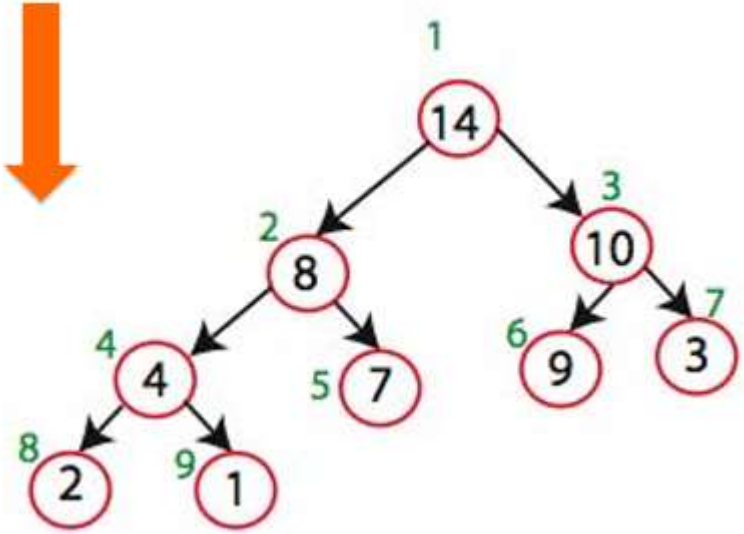
Swap A[10] and A[1]

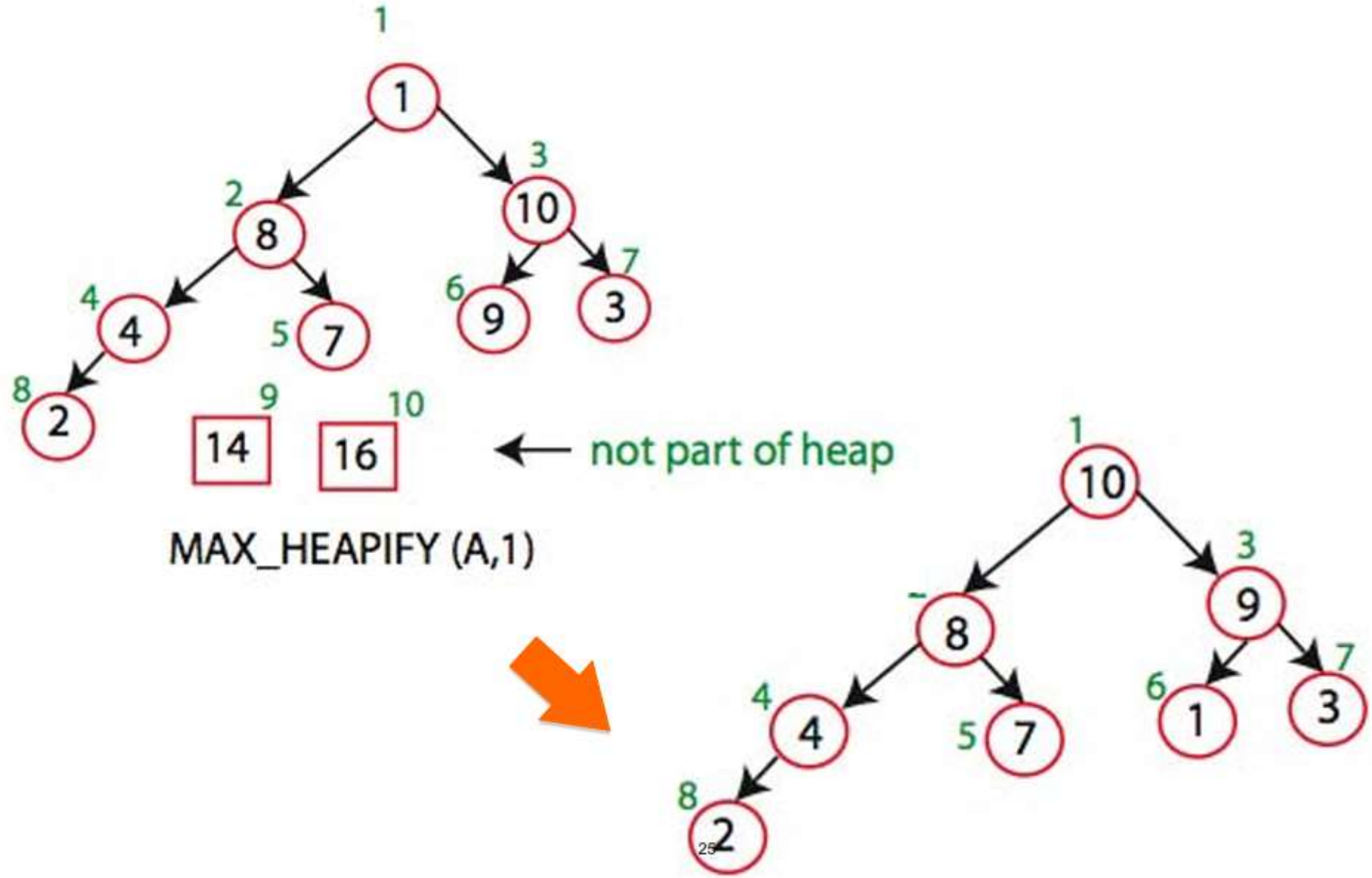
heap_size = 9



← not part of heap

Max_heapify(A,1)




1
1
2
8
3
10
4
4
5
7
6
9
7
3
8
2
9
14
10
16
not part of heap
MAX_HEAPIFY (A,1)
1
10
3
9
8
4
4
5
7
6
1
7
3
8
2

15. 【2013 统考真题】已知两个长度分别为 m 和 n 的升序链表，若将它们合并为长度为 $m+n$ 的一个降序链表，则最坏情况下的时间复杂度为（ ）。

A. $O(n)$　　B. $O(mn)$　　C. $O(\min(m, n))$　　D. $O(\max(m, n))$

15. D

考虑两个升序链表合并，两两比较表中元素，每比较一次，确定一个元素的链接位置（取较小元素，头插法）。当一个链表比较结束后，将另一个链表的剩余元素插入即可。最坏的情况是两个链表中的元素依次进行比较，因为 $2\max(m, n) \geqslant m+n$，所以时间复杂度为 $O(\max(m, n))$。此外，每次比较把两个链表中的较小结点插入新链表的表头，直到一个链表为空，因为原链表是升序排列的，要求合并后为降序排列的，因此还要把另一个链表剩下的结点一一插入新链表的表头，不论是最好情况还是最坏情况，都需要遍历两个链表中的所有结点。

19. 【2022 统考真题】使用二路归并排序对含 n 个元素的数组 M 进行排序时，二路归并排序的功能是（　）。

A. 将两个有序表合并为一个新的有序表

B. 将 M 划分为两部分，两部分的元素个数大致相等

C. 将 M 划分为 n 个部分，每个部分中仅含有一个元素

D. 将 M 划分为两部分，一部分元素的值均小于另一部分元素的值

19. A。

送分题。本书对归并的定义原话是“归并的含义是将两个或两个以上的有序表合并成一个新的有序表”，而二路归并是将两个有序表合并为一个新的有序表。

18. 【2021 统考真题】设数组 S[]={93,946,372,9,146,151,301,485,236,327,43,892}，采用最低位优先（LSD）基数排序将 S 排列成升序序列。第一趟分配、收集后，元素 372 之前、之后紧邻的元素分别是（ ）。

A. 43, 892　　B. 236, 301　　C. 301, 892　　D. 485, 301

18. C

基数排序是一种稳定的排序算法。由于采用最低位优先（LSD）的基数排序，即第一趟对个位进行分配和收集操作，因此第一趟分配和收集后的结果是{151, 301, 372, 892, 93, 43, 485, 946, 146, 236, 327, 9}，元素 372 之前、之后紧邻的元素分别是 301 和 892。

【2017统考真题】下列排序算法中，若将顺序存储更换为链式存储，则算法的时间效率会降低的是（ ）。

I. 插入排序 II. 选择排序 III. 冒泡排序 IV. 希尔排序 V. 堆排序

A. 仅I、II B. 仅II、III C. 仅III、IV D. 仅IV、V

17. D

在顺序存储的条件下，插入排序、选择排序、冒泡排序的时间复杂度都是 $O(n^2)$，更换为链式存储后的时间复杂度还是 $O(n^2)$。希尔排序和堆排序都利用了顺序存储的随机访问特性，而链式存储不支持这种性质，所以时间复杂度会增加。

【2020 统考真题】对大部分元素已有序的数组排序时，直接插入排序比简单选择排序效率更高，其原因是（ ）。

I. 直接插入排序过程中元素之间的比较次数更少

II. 直接插入排序过程中所需的辅助空间更少

III. 直接插入排序过程中元素的移动次数更少

A. 仅 I　　B. 仅 III　　C. 仅 I、II　　D. I、II 和 III

19. A

考虑比较极端的情况，对于有序数组，直接插入排序的比较次数为 $n-1$，简单选择排序的比较次数始终为 $1+2+\cdots+n-1=n(n-1)/2$，说法 I 正确。两种排序算法的辅助空间都为 $O(1)$，无差别，说法 II 错误。初始有序时，移动次数均为 0；对于通常情况，直接插入排序每趟插入都需要依次向后挪位，而简单选择排序只需与找到的最小元素交换位置，后者的移动次数少很多，说法 III 错误。

【2022 统考真题】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（ ）。

I. 大部分元素已有序　　II. 待排序元素数量很少

III. 要求空间复杂度为 $O(1)$　　IV. 要求排序算法是稳定的

A. 仅 I、II　　B. 仅 III、IV　　C. 仅 I、II、IV　　D. I、II、III、IV

20. D

直接插入排序和快速排序的特点如下表所示。

	适合初始序列情况	适合元素数量	空间复杂度	稳定性
直接插入排序	大部分元素有序	较少	$O(1)$	稳定
快速排序	基本无序	较多	$O(\log_2 n)$	不稳定

可见，说法 I、II、III、IV 都是采用直接插入排序而不采用快速排序的可能原因。

21.【2023 统考真题】下列排序算法中，不稳定的是（ ）。

I. 希尔排序　　II. 归并排序　　III. 快速排序　　IV. 堆排序　　V. 基数排序

A. 仅 I、II　　B. 仅 II、V　　C. 仅 I、III、IV　　D. 仅 III、IV、V

21. C

稳定的内部排序算法：插入排序、冒泡排序、归并排序和基数排序。不稳定的内部排序算法：简单选择排序、快速排序、希尔排序和堆排序。

【2013 统考真题】已知三叉树 T 中 6 个叶结点的权分别是 2, 3, 4, 5, 6, 7，T 的带权（外部）路径长度最小是（ ）。

A. 27　　B. 46　　C. 54　　D. 56

13. B

题中的三叉树为使 WPL 最小，必须构造三叉哈夫曼树，应满足$(n_0-1)\%(3-1)=0$ 的条件，因此需添加 1 个权值为 0 的虚叶结点，说明 7 个叶结点刚好可构成一棵严格的三叉树。按照哈夫曼树的原则，权为 0 的叶结点应离树根最远，构造三叉哈夫曼树的过程如下：

① 合并权值最小的三个结点 0, 2, 3，得到新结点的权值 =5，剩下 5, 4, 5, 6, 7。

② 合并权值最小的三个结点 4, 5, 5，得到新结点的权值 =14，剩下 14, 6, 7。

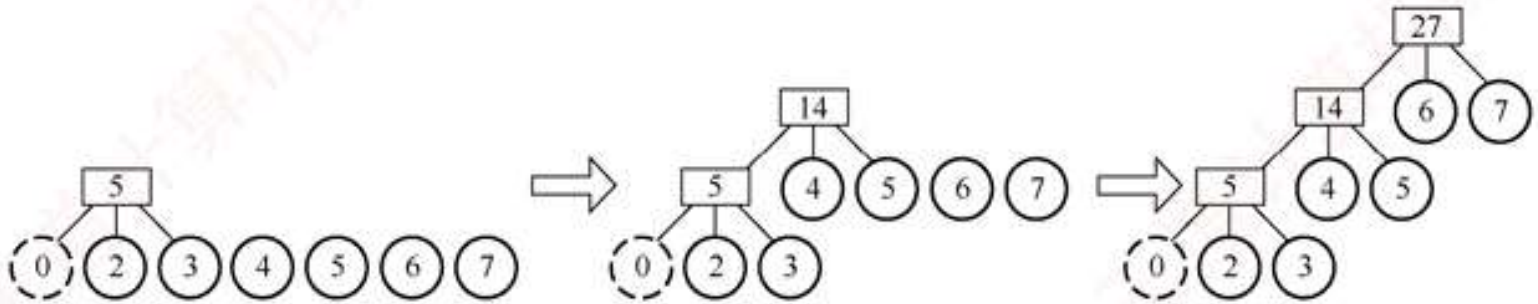
③ 合并权值最小的三个结点 6, 7, 14，得到新结点的权值 =27，仅有 27，建树完成。

$$\mathrm{WPL}=\sum_{1}^{n}\text{权值}W_{\text{叶结点}i}\times\text{深度}D_{\text{叶结点}i}=(2+3)\times3+(4+5)\times2+(6+7)\times1=46$$

或

$$\mathrm{WPL}=\sum_{1}^{m}\text{权值}W_{\text{分支结点}j}=27+14+5=46$$

每个分支结点的权值都累加了其下面所有分支结点的权值，因此采用第二种方法更方便。



16. 【2019 统考真题】设外存上有 120 个初始归并段，进行 12 路归并时，为实现最佳归并，需要补充的虚段个数是（ ）。

A. 1　　B. 2　　C. 3　　D. 4

16. B

在 12 路归并树中只存在度为 0 和度为 12 的结点，设度为 0 的结点数、度为 12 的结点数和要补充的结点数分别为 n_0、n_{12} 和 $n_{补}$，则有 $n_0 = 120 + n_{补}$，$n_0 = (12-1)n_{12} + 1$，可得 $n_{12} = (120 - 1 + n_{补})/(12-1)$。因为结点数 n_{12} 为整数，所以 $n_{补}$ 是使上式整除的最小整数，求得 $n_{补} = 2$。

此外，题中为实现最佳归并，应满足 12 叉哈夫曼树，$n = 120$，$m = 12$，不满足 $(n-1)\%(m-1) = 0$ 的条件，因此需要添加两个权值为 0 的叶结点，使得 $n = 121$，才能满足条件。